

Week 5: Perceptron Learning with Binary Cross-Entropy

CSCE 5218 – Deep Learning

Name: Zack Butz

Date: September 14, 2026

Given

For training example k , use

$$i_k = \sum_{i=0}^n w_i x_{ki} = \mathbf{w}^T \mathbf{x}_k, \quad y_k = \sigma(i_k) = \frac{1}{1 + e^{-i_k}},$$

where $t_k \in \{0, 1\}$, $x_{k0} = 1$, and

$$L_k = -[t_k \ln(y_k) + (1 - t_k) \ln(1 - y_k)].$$

1. Loss derivative

Question. Derive $\partial L_k / \partial y_k$ and simplify it to one fraction.

1. Expand the negative sign and differentiate each logarithm term with respect to y_k .

$$L_k = -t_k \ln(y_k) - (1 - t_k) \ln(1 - y_k)$$

$$\frac{\partial L_k}{\partial y_k} = -\frac{t_k}{y_k} + \frac{1 - t_k}{1 - y_k}$$

2. Put the terms over the common denominator $y_k(1 - y_k)$.

$$\frac{\partial L_k}{\partial y_k} = -\frac{t_k(1 - y_k)}{y_k(1 - y_k)} + \frac{(1 - t_k)y_k}{y_k(1 - y_k)}$$

$$\frac{\partial L_k}{\partial y_k} = \frac{-t_k(1 - y_k) + (1 - t_k)y_k}{y_k(1 - y_k)}$$

3. Show how the numerator simplifies.

$$-t_k(1 - y_k) + (1 - t_k)y_k = -t_k + t_k y_k + y_k - t_k y_k = y_k - t_k$$

4. Substitute the simplified numerator back into the fraction.

$$\boxed{\frac{\partial L_k}{\partial y_k} = \frac{y_k - t_k}{y_k(1 - y_k)}}$$

2. Parameter gradients

Question. Use the chain rule to derive $\partial L_k / \partial w_j$ for $j = 0, \dots, n$. Show every local derivative.

Use the computational path $w_j \rightarrow i_k \rightarrow y_k \rightarrow L_k$.

1. Write the chain rule.

$$\frac{\partial L_k}{\partial w_j} = \frac{\partial L_k}{\partial y_k} \frac{\partial y_k}{\partial i_k} \frac{\partial i_k}{\partial w_j}$$

2. Fill in the three local derivatives.

$$\frac{\partial L_k}{\partial y_k} = \frac{y_k - t_k}{y_k(1 - y_k)} \quad \frac{\partial y_k}{\partial i_k} = y_k(1 - y_k) \quad \frac{\partial i_k}{\partial w_j} = x_{kj}$$

3. Substitute them and explicitly cancel common factors.

$$\frac{\partial L_k}{\partial w_j} = \left(\frac{y_k - t_k}{y_k(1 - y_k)} \right) (y_k(1 - y_k)) x_{kj}$$

$$\frac{\partial L_k}{\partial w_j} = (y_k - t_k) x_{kj}.$$

4. Explain why the same formula includes the bias. What happens when $j = 0$ and $x_{k0} = 1$?
The formula includes the bias so that the model can classify data which is not separable through the origin acting like a boundary condition for the formula. When $j = 0$ and $x_{k0} = 1$, the gradient is simply $y_k - t_k$.

3. Gradient-descent update

Question. Write one update rule that applies to every w_j , including w_0 , using learning rate η .

1. Start from gradient descent:

$$w_j \leftarrow w_j - \eta \frac{\partial L_k}{\partial w_j}.$$

2. Substitute your simplified gradient.

$$w_j \leftarrow w_j - \eta(y_k - t_k)x_{kj}$$

3. When $x_{kj} > 0$, explain what happens to w_j when $t_k - y_k > 0$, and when $t_k - y_k < 0$.
When $x_{kj} > 0$ and $t_k - y_k > 0$, the update is positive, so w_j increases. When $t_k - y_k < 0$, the update is negative, so w_j decreases.

4. One numerical update

Use

$$\mathbf{x}_k = (1, 1, 2), \quad \mathbf{w} = (0, 0.2, -0.1), \quad t_k = 1, \quad \eta = 0.1.$$

Report nonlinear values to four decimal places.

4.1 Forward pass before the update

$$i_k = w_0 x_{k0} + w_1 x_{k1} + w_2 x_{k2} = \mathbf{w}^T \mathbf{x}_k$$

$$y_k = \sigma(i_k) = \frac{1}{1 + e^{-i_k}}$$

$$L_k = -[t_k \ln(y_k) + (1 - t_k) \ln(1 - y_k)]$$

$$\frac{\partial L_k}{\partial w_j} = (y_k - t_k) x_{kj}.$$

$$w_j \leftarrow w_j - \eta(y_k - t_k) x_{kj}$$

4.2 Gradients

$$\frac{\partial L_k}{\partial w_0} = \left(\frac{1}{1 + e^0} - 1\right)1 \quad \frac{\partial L_k}{\partial w_1} = \left(\frac{1}{1 + e^0} - 1\right)1 \quad \frac{\partial L_k}{\partial w_2} = \left(\frac{1}{1 + e^0} - 1\right)2$$

$$\frac{\partial L_k}{\partial w_0} = -\frac{1}{2} \quad \frac{\partial L_k}{\partial w_1} = -\frac{1}{2} \quad \frac{\partial L_k}{\partial w_2} = -1$$

4.3 Updated parameters

$$w_0^{\text{new}} = 0 - 0.1(-0.5) = 0.05 \quad w_1^{\text{new}} = 0.2 - 0.1(-0.5) = 0.25 \quad w_2^{\text{new}} = -0.1 - 0.1(-1) = 0$$

4.4 Forward pass after the update

$$i_k^{\text{new}} = (0.05)(1) + (0.25)(1) + (0)(2) = 0.3$$

$$y_k^{\text{new}} = \sigma(0.3) = \frac{1}{1 + e^{-0.3}} \approx 0.5744$$

$$L_k^{\text{old}} = -[1 \ln(0.5) + (1 - 1) \ln(1 - 0.5)] = -\ln(0.5) \approx 0.6931$$

$$L_k^{\text{new}} = -[1 \ln(0.5744) + (1 - 1) \ln(1 - 0.5744)] = -\ln(0.5744) \approx 0.5544$$

Briefly verify whether the loss moved in the expected direction. The loss decreased from $L_k \approx 0.6931$ to $L_k^{\text{new}} \approx 0.5544$, which is the expected direction since the value of y_k is closer to its expected value of 1 after the update.

5. Comparison with squared error

Question. For a sigmoid neuron, why does squared error retain $y_k(1 - y_k)$ in its weight gradient while BCE does not? What can happen near sigmoid outputs of 0 or 1?

1. If $E_k = \frac{1}{2}(t_k - y_k)^2$, write its chain-rule gradient with respect to w_j .

$$\frac{\partial E_k}{\partial w_j} = \frac{\partial E_k}{\partial y_k} \frac{\partial y_k}{\partial i_k} \frac{\partial i_k}{\partial w_j}$$

$$\frac{\partial E_k}{\partial y_k} = -(t_k - y_k) = y_k - t_k$$

$$\frac{\partial y_k}{\partial i_k} = y_k(1 - y_k)$$

$$\frac{\partial i_k}{\partial w_j} = x_{kj}$$

$$\frac{\partial E_k}{\partial w_j} = (y_k - t_k)y_k(1 - y_k)x_{kj}$$

2. Explain one training consequence when y_k is close to 0 or 1.

When y_k is close to 0 or 1, the term $y_k(1 - y_k)$ becomes very small, which can lead to attenuated gradients. This means that the weight updates will be very small, slowing down the learning process and making it difficult for the model to converge to an optimal solution. This is one of the reasons why BCE is preferred over squared error for classification tasks, as it avoids this issue and provides more stable gradients due to removing the extra sigmoid derivative factor seen in section 2 part 3.

AI assistance disclosure

Initial bootstrapping for this homework was provided by ChatGPT. All subsequent work, including derivations, calculations, and explanations, was completed by me.